



廣東財經大學
GUANGDONG UNIVERSITY OF FINANCE & ECONOMICS

《Python 自然语言处理》

课程报告

院 系	信息学院
学 号	21251106142
姓 名	严阳
班 级	计实一班
班级序号	13
学 期	2023-2024 学年第 1 学期
校 区	校本部
任课教师	冯俊健
阅卷教师	冯俊健
交稿日期	2024 年 6 月 10 日

目 录

基于微调 Bert 模型和 LDA 模型的商品评论分析.....	1
1 摘要&关键词	1
2 绪论	1
2.1 分组和分工情况.....	1
2.2 背景与意义	1
2.3 存在研究问题.....	2
2.4 实现功能	3
3 研究现状	3
3.1 文献搜索与评述.....	3
3.2 引用文献描述.....	3
4 概念与原理	4
4.1 情感分析	4
4.2 共现语义网络.....	5
4.3 主题模型	5
4.4 信息爬取	7
5 算法介绍	7
5.1 Bert.....	7
5.2 共现语义网络.....	8
5.3 LDA	9
5.4 信息爬取	11
6 系统界面与实验	13
6.1 系统简介	13
6.2 ONNX 模型部署.....	14
6.2.1 Bert 微调与推理.....	14
6.2.2 Bert 转 ONNX 模型与推理	15
6.3 实验结果与分析.....	16
6.3.1 条形码识别与信息提取.....	16
6.3.2 商品信息评论获取.....	16
6.3.3 商品评论情感分析.....	18
6.3.4 评论数据分析.....	19
7 总结与展望	20
7.1 系统实现的总结.....	20
7.2 系统功能的展望.....	20
7.3 个人成长的规划.....	21
8 参考文献	22
9 附录	23
9.1 条形码检测识别.....	23
9.2 商品信息评论获取.....	26
9.3 商品评论情感分析.....	29
9.4 商品评论数据分析.....	31

基于微调 Bert 模型和 LDA 模型的商品评论分析

1 摘要&关键词

摘要

本文提出并实现了一种用于商品评论分析的自动化综合框架，先通过条形码图像检测和识别并检索商品信息，然后通过爬虫的模拟登录技术获取商品在电商平台的评论，再通过获取的评论数据微调训练 Bert 深度学习 NLP 模型，并转化成 ONNX 模型进行推理区分商品评论的情感，最后通过机器学习算法对情感数据进行数据分析和可视化，展现词频词云、共现语义网络和 LDA 模型的评论主题分析。

关键词

商品评论分析，图像检测和识别，爬虫，BERT，LDA

2 绪论

2.1 分组和分工情况

组长：严阳

工作内容：系统框架概念的阐述，条形码检测和识别功能开发，爬取商品评论功能开发，Bert 模型的微调与推理，词频分析与词云生成，共现语义网络的实现，LDA 建模与话题分析的实现

组员：雷袁

工作内容：系统综合性能和可靠性检测，测试数据准备的准备，部分功能模块扩展逻辑的实现，模拟登录功能实现，Bert 模型转为 ONNX 模型并推理，数据分析合理性探究。

2.2 背景与意义

背景

在信息时代，随着电子商务的快速发展，消费者在网上购物时，越来越依赖于其他用户的评论和反馈。这些评论不仅为潜在消费者提供了重要的参考信息，还为商家改进产品和服务提供了宝贵的反馈。然而，面对海量的评论信息，人工分析和处理显得既费时又费力，因此，机器学习是时代的工具利用自然语言处理（NLP）技术进行自动化评论分析变得尤为重要。

机器学习作为人工智能的重要分支，已经在文本分类、情感分析等领域展现出强大的能力。通过学习大量的标注数据，机器学习模型能够自动提取特征并进行预测，从

而实现高效准确的文本处理。在文本分析中，主题模型（如 LDA, Latent Dirichlet Allocation）被广泛应用于提取文本中的潜在主题。LDA 是一种生成模型，通过假设文档是由若干个主题混合生成的，每个主题由词语分布生成，能够帮助理解文本的潜在结构。

BERT 作为一种先进的预训练语言模型，自其发布以来，在多项自然语言处理任务中均表现出色。BERT 通过双向编码器架构，能够从大规模未标注的文本数据中学习通用的语言表示，再通过微调适应具体任务。其在情感分析、问答系统等任务中取得了显著的成果，展现了深度学习技术在文本处理中的强大能力。

意义

提升评论分析效率：传统的评论分析通常依赖于人工，效率低下且容易受到主观因素的影响。利用 BERT 模型进行评论情感分析，可以实现自动化处理，大幅提升效率和准确性。

优化消费者体验：通过自动化分析大量用户评论，商家能够迅速了解产品或服务的优缺点，及时作出调整和改进，从而提升消费者的购物体验和满意度。

支持决策：精确的情感分析有助于商家进行市场分析、产品改进和营销策略制定。例如，通过识别出积极或消极的评论，商家可以更好地了解市场需求和消费者偏好。

技术创新：本项目不仅展示了 BERT 在评论情感分析中的应用，还包括了模型的微调、预测和转换为 ONNX 格式以提高推理效率。通过这些技术创新，可以更好地满足实际应用中的性能需求。

可扩展性：本项目的框架和方法不仅适用于电子商务评论分析，还可以扩展到其他领域的文本情感分析，如社交媒体评论、新闻评论等，从而具有广泛的应用前景。

2.3 存在研究问题

反爬机制对于爬虫的影响：尽管在很多地方都针对反爬机制的处理，但是较大的电商平台仍存在一些难以避免的反爬措施，虽然能顺利爬取，但是爬取的信息数量被限制。

商品信息的局限性：由于是在一些信息收录平台上，根据条形码进行商品信息获取，如果商品没有在此平台收录将会查询不到信息。

主题模型的局限性：语义理解不足，传统的主题模型（如 LDA）在捕捉词语间复杂的语义关系方面存在局限，尤其在处理短文本和噪声数据时，效果较差；动态适应性差，随着时间推移，用户的兴趣和评论内容会发生变化，现有的主题模型难以动态适应这种变化。

深度学习模型的需求与挑战：

数据需求，深度学习模型（如 BERT）需要大量标注数据进行训练，而在很多应用

场景下，标注数据往往稀缺且获取成本高；计算资源，训练和部署深度学习模型需要大量的计算资源，对于小型企业或资源受限的应用场景，难以实现。

模型的可解释性和透明度：黑箱问题，深度学习模型虽然表现优异，但其内部机制复杂，缺乏可解释性，这在某些需要决策透明的领域（如医疗、法律）是一个重大问题；用户信任，由于缺乏可解释性，用户对模型预测结果的信任度可能不足，影响其在实际应用中的接受度和推广。

跨领域的通用性：

泛化能力，现有模型在特定领域的表现可能良好，但在跨领域应用时，往往表现不佳，缺乏足够的泛化能力；适应性，不同领域的数据特征和语言表达方式存在显著差异，如何提高模型在跨领域应用中的适应性是一个重要挑战。

2.4 实现功能

条形码的检测和识别
模拟登录并爬取商品信息和商品评论
微调 Bert 并且转化为 ONNX 模型进行评论情感分析推理
词频统计
共现语义网络
LDA 建模主题分析

3 研究现状

3.1 文献搜索与评述

情感分析是自然语言处理（NLP）中的一个重要领域，旨在通过计算机分析文本中的情感倾向。

主题模型旨在从大量文档中自动发现主题。

3.2 引用文献描述

Pang et al. (2002)

问题：自动化提取电影评论中的情感信息。
方法：使用朴素贝叶斯、最大熵分类和支持向量机进行情感分类。
效果：机器学习技术有效，但在处理复杂句法结构和隐含情感时存在挑战。

Devlin et al. (2018)

问题：提高情感分析模型的准确性和泛化能力。
方法：BERT 模型，使用双向 Transformer 架构进行预训练和微调。
效果：BERT 在多个 NLP 任务（包括情感分析）中取得最佳成绩，提高了模型理解

和生成能力。

Blei et al. (2003)

问题：从文档集中提取主题。

方法：潜在狄利克雷分配（LDA）模型，通过统计方法发现潜在主题。

效果：LDA 模型在文本主题提取中表现良好，能够从大规模文档集中提取有意义的主题。

Mimno et al. (2011)

问题：提高 LDA 模型在大规模文档集上的效率。

方法：基于在线变分推断的 LDA 模型，提高大规模数据集的主题建模效率。

效果：显著提高 LDA 模型处理大规模文档集的效率，验证了其有效性。

Zhao et al. (2015)

问题：在短文本数据上进行有效的主题建模。

方法：基于聚类 and 主题模型的混合方法，引入上下文信息改善主题提取效果。

效果：该方法在短文本主题建模中表现优异，特别是在社交媒体数据分析中效果良好。

4 概念与原理

4.1 情感分析

情感分析是自然语言处理（NLP）的一个重要任务，旨在确定文本中的情感极性，即文本是表达积极、消极还是中性的情感。以下是一些关键概念和技术：

Bert 模型：

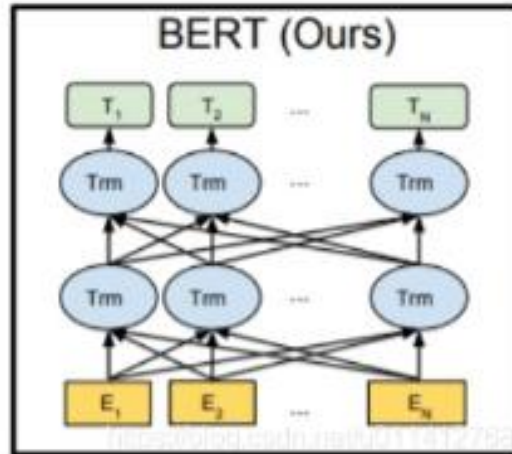
Bert 通过双向编码器同时考虑文本的左上下文和右上下文，能够更好地理解词语在上下文中的含义。

公式（自注意力）：

$$Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

其中， Q 是查询矩阵， K 是键矩阵， V 是值矩阵， d_k 是键的维度。

Bert 图：



4.2 共现语义网络

共现语义网络是一种通过分析文本中词语的共现关系来揭示词语之间语义关联的技术。共现关系指的是在同一语境中出现的词语对。通过构建词语共现矩阵，并将其可视化网络图，我们可以直观地观察词语之间的语义联系。

公式：

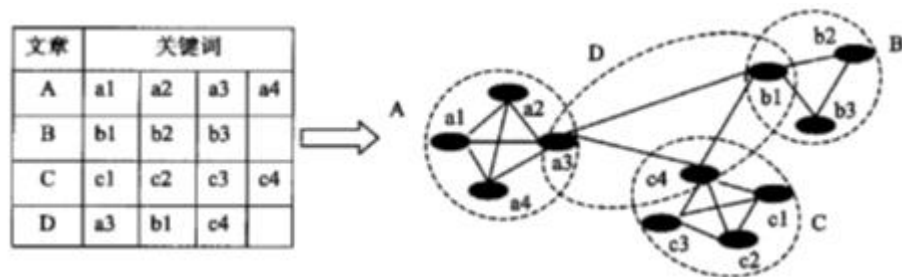
$$M_{ij} = \sum_{d \in D} \sum_{(w_a, w_b) \in P(d)} \delta(w_a = w_i \wedge w_b = w_j)$$

D 表示文档集合。

$P(d)$ 表示文档 d 中所有相邻词语对。

$\delta(\cdot)$ 是指示函数，当括号内条件为真时取值为 1，否则为 0。

共现原理图：



4.3 主题模型

主题模型用于从文档集中发现隐含主题，常用的方法包括潜在狄利克雷分布(LDA)。先使用 TF-IDF 分析词频，在进行 LDA 建模拟合生成主题，然后根据每个主题内容当中的核心词频绘制词云。

TF-IDF：

词在一个文档的频率+词出现在整个语料库文档的情况，用以评估一字词对于一个

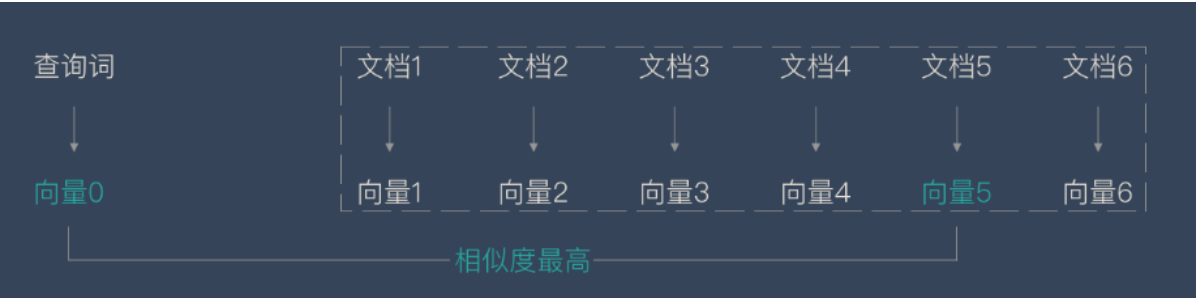
文件集或一个语料库中的其中一份文件的重要程度。

公式：

$$TFIDF = tf(w, d) \times \log \frac{N}{df(w)}$$

其中， $tf(w, d)$ 是词 w 在文档 d 中的词频， $df(w)$ 是包含词 w 的文档数量， N 是文档总数。

TF-IDF 图：



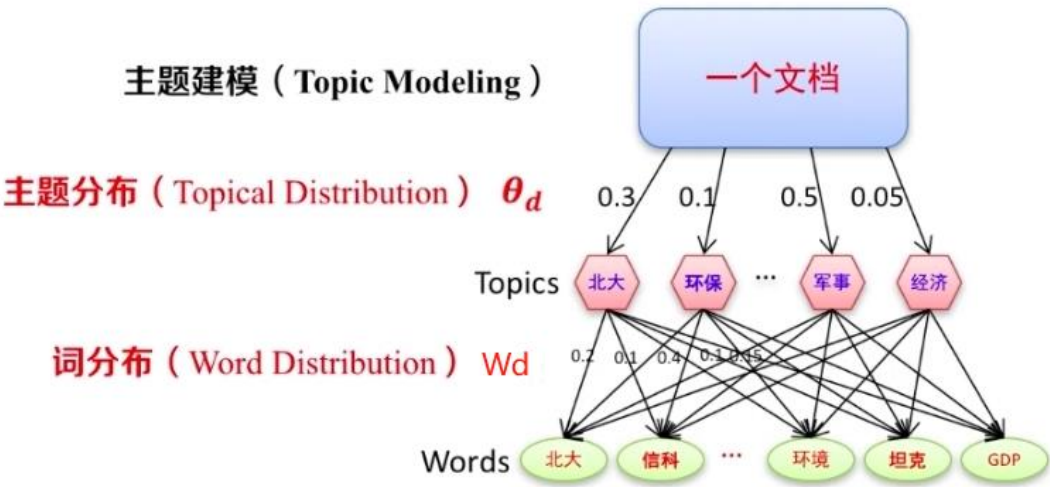
潜在狄利克雷分布（LDA）：一种生成式概率模型，通过词语和文档的共现关系来发现文档中的主题。

公式：

$$p(\mathbf{w} | \alpha, \beta) = \int_{\theta} \left(\prod_{d=1}^D \left(\int_{z_d} \left(\prod_{n=1}^{N_d} p(w_{dn} | z_{dn}, \beta) p(z_{dn} | \theta_d) \right) p(\theta_d | \alpha) \right) \right) d\theta$$

其中， w 是文档集， α 和 β 是狄利克雷分布的超参数，前者控制文档主题丰富度，后者控制主题中包含词语丰富度， θ_d 是文档 d 的主题分布， z_{dn} 是文档 d 中第 n 个词的主题分配， w_{dn} 是文档 d 中的第 n 个词

LDA 图：



4.4 信息爬取

Selenium 模拟登录：由于反爬措施，电商平台会限制匿名用户使用，通过模拟登录加上开启浏览器的代理调试功能能有效避免登录限制。

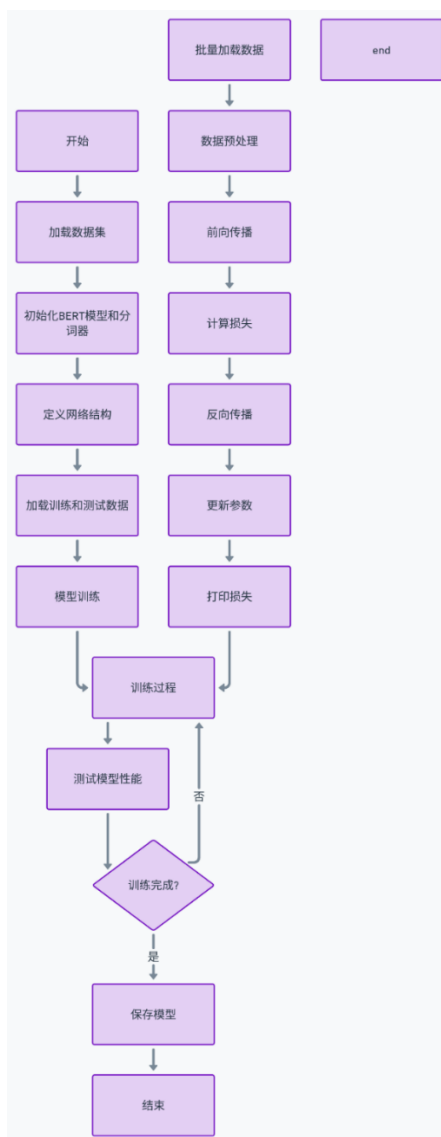
定位 html 元素并解析 json 数据：通过定位 ID 标签的位置能准确获取指定数据，进行静态和动态的网页信息获取。

仿真操作：通过设定延迟运行时间，模仿真人操作，避免服务器判定为非法访问对 IP 进行封锁。

5 算法介绍

5.1 Bert

逻辑图：



详细步骤:

1. 数据预处理:

- 加载训练和测试数据集。
- 将数据集转换为 DataFrame 格式便于处理。

2. BERT 模型加载与微调:

- 使用预训练的 BERT 模型和分词器。
- 自定义网络结构 **Net**, 在 BERT 的基础上添加全连接层和 Sigmoid 激活函数用于二分类任务。
- 使用 Adam 优化器和学习率调度器进行优化。

3. 模型训练:

- 在训练过程中冻结 BERT 参数, 仅对自定义的全连接层进行训练。
- 通过前向传播计算损失, 反向传播更新模型参数。
- 在训练过程中定期测试模型性能并保存模型。

4. 模型测试:

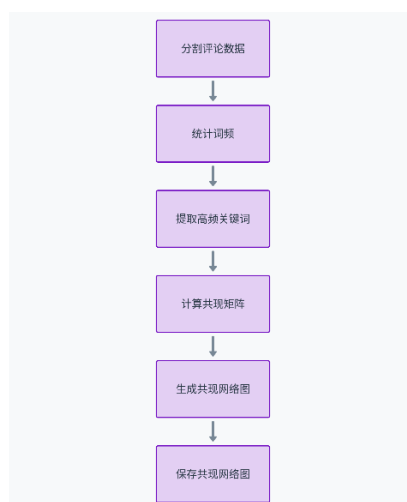
- 在测试集上评估模型的性能。
- 计算分类报告、准确率和 AUC 值。

5. 辅助功能:

- 创建保存模型的目录。

5.2 共现语义网络

逻辑图:



详细步骤:

1. 词频统计:

- 首先, 对文本进行分词, 得到每个文档中的词语列表。
- 统计每个词语在文档集中出现的频次, 筛选出高频词。

2. 共现矩阵:

- 构建词语共现矩阵, 用于表示词语之间的共现频次。矩阵的行和列表示词语, 矩阵中的值表示词语对在文档中同时出现的次数。

3. 共现关系计算:

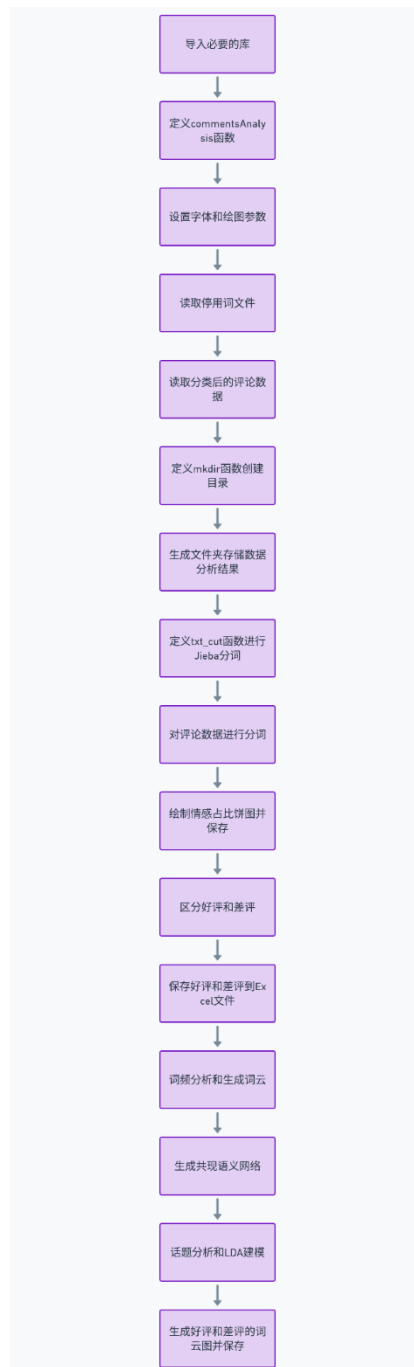
- 遍历每篇文档,统计每对高频词在同一文档中相邻出现的次数,填充共现矩阵。

4. 可视化:

- 利用网络图可视化工具(如 NetworkX),将共现矩阵转化为网络图,节点表示词语,边表示共现关系,边的权重表示共现次数。

5.3 LDA

逻辑图:



详细步骤:

1. 数据预处理

分词和去停用词: 使用 Jieba 库对评论文本进行中文分词, 将句子分割成词语列表。从分词结果中去除停用词, 停用词是一些高频但对文本主题贡献不大的词, 如“的”、“了”、“是”等。

词袋模型: 使用词袋模型将分词后的文本转换成矩阵形式, 其中行表示评论, 列表示词语, 矩阵中的值表示词语在评论中的出现次数。

2. TF-IDF 转换

将词频矩阵转换为 TF-IDF (Term Frequency-Inverse Document Frequency) 矩阵。

TF-IDF 可以降低高频词的权重, 突出少见但重要的词。

使用 TfidfVectorizer 将评论文本转换为 TF-IDF 矩阵。

3. LDA 模型训练

设置主题数量: 选择需要生成的主题数量 K (比如 8 个主题), 这个数值需要根据具体情况调整。

模型训练: 使用 Latent Dirichlet Allocation (LDA) 对 TF-IDF 矩阵进行建模。LDA 是一种生成模型, 它假设每个文档是由多个主题生成的, 每个主题由一组词语以不同的概率组成。

通过最大化似然估计来训练 LDA 模型, 使其能够找到最能解释文档集的数据分布。

4. 主题关键词提取

提取每个主题的关键词: 从每个主题中提取具有最高权重的前 N 个词语, 这些词语代表该主题的主要内容。

输出主题和关键词: 打印或保存每个主题对应的关键词及其权重, 这有助于理解每个主题的具体内容。

5. 主题标签分配

计算每条评论的主题分布: 通过 LDA 模型计算每条评论属于各个主题的概率分布。

分配主题标签: 将每条评论分配到概率最高的主题, 这样每条评论都有一个最可能的主题标签。

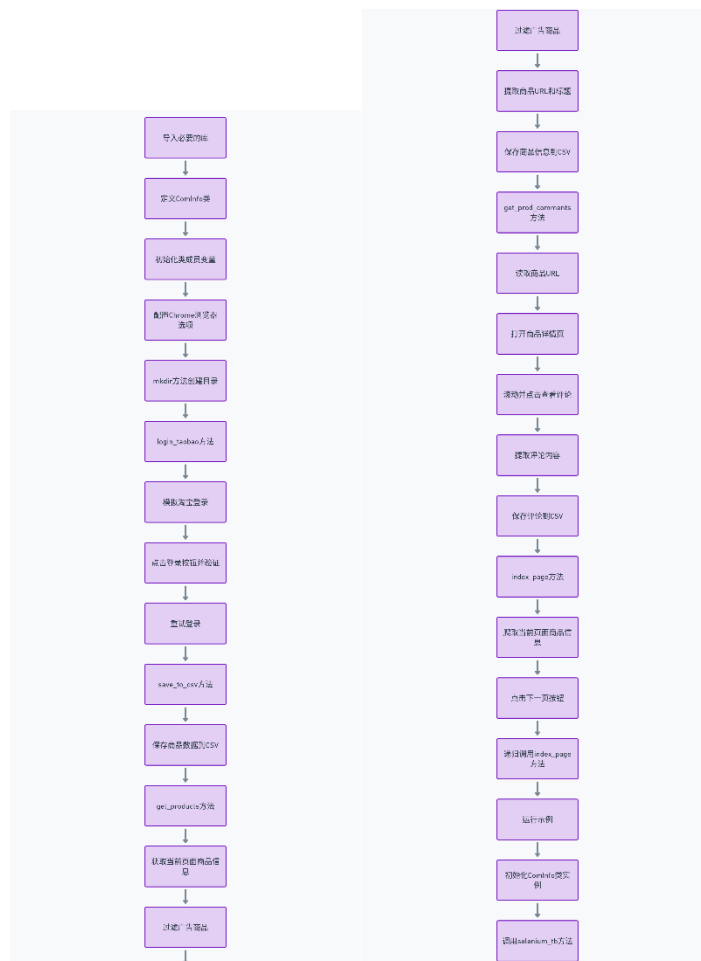
6. 结果可视化

生成词云图: 使用 WordCloud 库生成每个主题的词云图, 通过词云图可以直观地展示每个主题的关键词及其权重。词云图中的词语大小和颜色表示词语的重要性和频率。

保存和展示结果: 将生成的词云图保存到指定目录。将每条评论及其对应的主题标签保存到文件中, 便于后续分析和查看。

5.4 信息爬取

逻辑图：



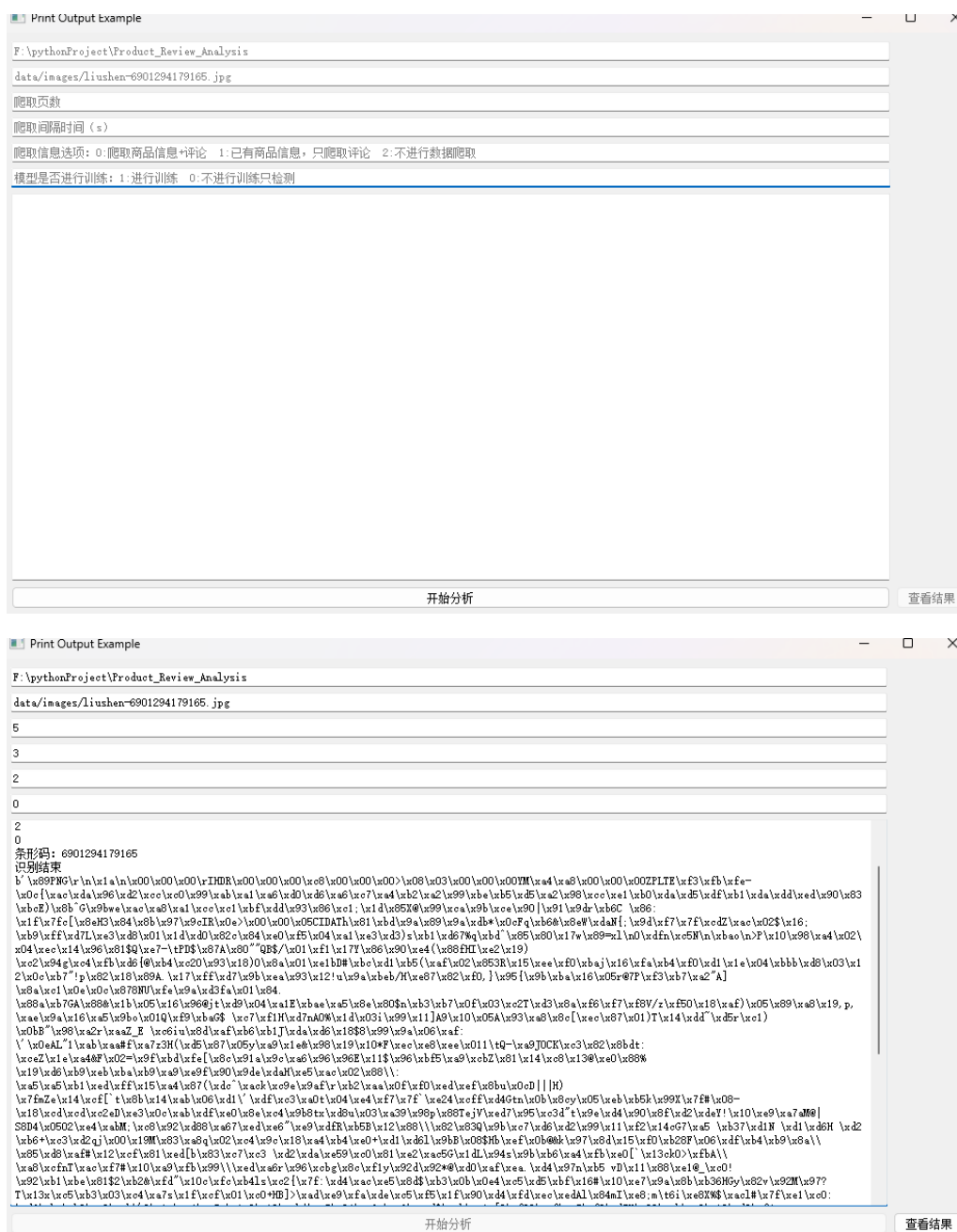
详细步骤：

1. **导入必要的库：**导入处理数据、操作文件、执行延时、进行网络爬虫等所需的库。
2. **定义 ComInfo 类：**
 - 初始化类成员变量，包括商品条形码、项目目录、Chrome 浏览器配置、关键字、当前页数、最大页数、等待时间等。
 - 配置 Chrome 浏览器选项，使其处于调试模式，并设置隐身模式及用户数据目录。
3. **mkdir 方法：**根据指定路径创建目录，如果目录不存在，则创建该目录；如果存在，则提示目录已存在。
4. **login_taobao 方法：**
 - 模拟淘宝登录过程，包括输入账号和密码。
 - 尝试点击登录按钮，验证登录是否成功。

- 如果登录失败，会进行多次重试。
5. **save_to_csv 方法：**将商品数据保存到 CSV 文件中，如果文件不存在，则写入表头；如果文件存在，则追加写入数据。
6. **get_products 方法：**
- 获取当前页面的商品信息。
 - 过滤掉广告商品。
 - 提取商品的 URL 和标题，并将这些信息保存到 CSV 文件中。
7. **get_prod_comments 方法：**
- 读取商品的 URL。
 - 对每个商品 URL，打开商品详情页。
 - 滚动至页面中的评价部分，并点击查看评论。
 - 提取评论内容，去除“追评”部分。
 - 将评论内容保存到 CSV 文件中。
8. **index_page 方法：**
- 爬取当前页面的商品信息。
 - 尝试点击下一页按钮。
 - 如果成功跳转到下一页，并且未超过最大页数，则递归调用 **index_page** 方法继续爬取下一页的信息。
9. **运行示例：**
- 初始化 **ComInfo** 类实例。
 - 调用 **selenium_tb** 方法，执行整个爬取过程，包括登录淘宝、获取商品信息及其评论。

6 系统界面与实验

6.1 系统简介



本系统能基本上实现了一键爬取指定商品的信息，先通过模块 BarCodeDet 和 BarCodeRec 进行条形码图片的检测和识别，获取到商品信息后，通过模块 ComInfo 中利用 Selenium 模拟登录和 chromedriver 代理进行商品信息和评论的爬取，并对商品评论进行情感分析（Bert-finetining 和 Bert-ONNX）和简单数据分析（共现语义网络、LDA 主题模型）。

6.2 ONNX 模型部署

6.2.1 Bert 微调与推理

预训练模型：

	precision	recall	f1-score	support
0	0.71	0.72	0.72	155
1	0.87	0.87	0.87	345
accuracy			0.82	500
macro avg	0.79	0.79	0.79	500
weighted avg	0.82	0.82	0.82	500
准确率：0.822				
AUC：0.88835904628331				

微调模型：

	precision	recall	f1-score	support
0	0.76	0.81	0.79	155
1	0.91	0.88	0.90	345
accuracy			0.86	500
macro avg	0.84	0.85	0.84	500
weighted avg	0.87	0.86	0.86	500
准确率：0.862				
AUC：0.9300607760635812				

commentsSentiment.csv	
70	1, 好用
71	1, 整体评价：) 没有收到货品，菜鸟官方客服已理赔，服务好
72	0, 和超市买的不一样
73	0, 还好，物美价廉
74	1, 整体评价：和之前买的一样 包装与外观：没毛病 使用效果：味道不好闻，没有塑料瓶包装味道好，驱蚊效果一般
75	1, 很好
76	1, 香水不错，只是包装不好，瓶盖坏了，到家都漏啦。
77	1, 六神的就是好效果棒，推荐购买
78	0, Good
79	1, 效果不错，老人就认老牌子
80	1, 正宗六神花露水195ml，确实地道，有效驱蚊，优质保证。非常感谢快递万东师傅于酷暑三伏天，周到服务送货在家中👍👍👍五星好评
81	1, 体验了一下可以休息
82	1, 六神花露水还是原来的味道，很喜欢，店家态度很好，用完还会再来
83	1, 宝贝很快收到了，今天才有空评价，还是小时候的那个味道，跟我在超市买的一样，送了小喷瓶，出门携带很方便，谢谢卖家

基本原理：

初始化：加载数据集、BERT 模型和分词器，设置训练超参数，并初始化训练和测试数据加载器。

数据集：采用的是 weibo2018 评论数据集。

微调方式：自定义一个网络（全连接层和 sigmoid 函数），在训练时候将 Bert 输出的特征传递给自定义网络，然后进行向前传播、损失计算和反向传播。

6.2.2 Bert 转 ONNX 模型与推理

Bert-ONNX 模型:

```
[input: "onnx::Gemm_0"
input: "fc.weight"
input: "fc.bias"
output: "/fc/Gemm_output_0"
name: "/fc/Gemm"
op_type: "Gemm"
attribute {
  name: "alpha"
  type: FLOAT
  f: 1
}
attribute {
  name: "beta"
  type: FLOAT
  f: 1
}
attribute {
  name: "transB"
  type: INT
  i: 1
}
, input: "/fc/Gemm_output_0"
output: "4"
name: "/sigmoid/Sigmoid"
op_type: "Sigmoid"
]
```

		precision	recall	f1-score	support
	0	0.76	0.81	0.79	155
	1	0.91	0.88	0.90	345
	accuracy			0.86	500
	macro avg	0.84	0.85	0.84	500
	weighted avg	0.87	0.86	0.86	500
	准确率: 0.862				
	AUC: 0.9300607760635812				

```
6923450656181\commentsSentiment_onnx.csv
10 1,挺好的
11 1,卖家服务态度很好,物流也很快,!
12 1,挺不错的
13 0,口感味道:
14 1,商品分量:好 口感味道:好
15 1,正品
16 0,口感味道:一不好
17 1,比便利店便宜多了
18 1,物美价廉
19 0,great
20 0,good
21 1,还没吃,但是包装可以,日期很好,快递员非常好,今天还下雨,送到门口,询问确认,我都没出门口,鞋也没粘湿
22 1,是正品 推荐购买
23 0,货不对板,拍之前明明有看清楚要嘛,结果收到货还是个错的
```

基本原理:

初始化 BertOnnxPerdict 类: 初始化模型和设备配置, 加载微调后的 PyTorch 模型和预训练 BERT 模型。加载评论数据并进行预处理。

导出 ONNX 模型: 将提取的特征输入到微调后的分类器模型中, 并使用 torch.onnx.export 将模型导出为 ONNX 格式。

模型预测: 在 bert_onnx_predicted 方法中, 遍历评论列表, 对每条评论进行分词和特征提取。使用 onnxruntime 加载 ONNX 模型并进行预测, 将结果保存到 CSV 文件中。

6.3 实验结果与分析

6.3.1 条形码识别与信息提取



```
条形码: 6923450656181
识别结果
欢迎使用ddddocr, 本项目专注带动行业内卷, 个人博客: wenanzhe.com
训练数据支持来源于: http://t46.56.284.113:19199/preview
爬虫框架feapder可快速一键接入, 快速开启爬虫之旅: https://github.com/Boris-code/feapder
谷歌reCaptcha验证码 / hCaptcha验证码 / funCaptcha验证码商业识别接口: https://yescaptcha.com/i/NSwk7i
00[000;09030m220002040-00060-00050 01040;02090;04000.090103000101040 0[0W0;00m0m0x0r0u0m0t010m0e0;0,0 0e0x0e0c0u0t0i0o0n0_0f0r0a0m0e0.0c0c0;0005080
0o0n0n0x0r0u0m0t0i0m0e0;0;0E0x0e0c0u0t0i0o0n0F0r0a0m0e0;0;0V0e0r0i0f0y000u0t0p0u0t005010z0e0s0]0 0E0x0p0e0c0t0e0d0 0s0h00a0p0e0 0f0r0o0m0
0m0o0d0e0l0 0o0f0 0{010,0-010}0 0d0o0e0s0 0m0o0t0 0m0a0t0c0h0 0a0c0t0u0a0l0 0s0h00a0p0e0 0o0f0 0{0200,010,08020100}0 0f0o0r0 0o0u0t0p0u0t0
030007000[0m]
2024-06-05 14:29:40,913 - 25472-MainThread - F:\pythonProject\Product_Review_Analysis\barcode_rec.py[line:73] - INFO: 当前验证码为 3141
2024-06-05 14:29:42,263 - 25472-MainThread - F:\pythonProject\Product_Review_Analysis\barcode_rec.py[line:78] - INFO: {'code': 1, 'msg': '查询成功',
'\json': {'code_id': '2144', 'code_sn': '6923450656181', 'code_type': '标前商品和罐头', 'code_name': '苗达木糖醇薄荷40粒瓶装56g', 'code_company': '玛氏新牌',
'\糖果(中国)有限公司', 'code_address': '中国', 'code_price': '9.20', 'code_spec': '瓶装', 'code_unit': '', 'code_img': '/Upload/201805/ypufg5p2aqq.jpg',
'\code_field1': '', 'code_pinpai': '', 'code_addtime': '2017-12-07 14:20:29'}, 'type': 'verify'}
2024-06-05 14:29:42,352 - 25472-MainThread - F:\pythonProject\Product_Review_Analysis\barcode_rec.py[line:49] - INFO: output[6923450656181] 创建成功
2024-06-05 14:29:42,353 - 25472-MainThread - F:\pythonProject\Product_Review_Analysis\barcode_rec.py[line:90] - INFO:
output[6923450656181]6923450656181.png
苗达木糖醇薄荷40粒瓶装56g 瓶装
信息获取结果
```

分析: 先通过 ssdlite320_mobilenet_v3_large 模型进行目标检测, 将图像转变为灰度图, 对于检测到的目标通过 pyzbar 库检测并且识别解码条形码。然后通过 dddocr 库解决条形码查询平台输入验证码问题, 最终获取条形码内容然后将商品信息进行爬取。

6.3.2 商品信息评论获取

```
已经登录
output[6923450656181] 目录已存在
正在爬取: https://s.taobao.com/search?page=1&q=%E7%9B%8A%E8%BE%E6%9C%A8%E7%B3%96%E9%86%87%E8%96%84%E8%8D%B740%E7%B2%92%E7%93%B6%E8%A3%8556q%20%E7%93%B6%E8%A3%85&tab=all
output[6923450656181] 目录已存在
正在爬取: https://s.taobao.com/search?page=2&q=%E7%9B%8A%E8%BE%E6%9C%A8%E7%B3%96%E9%86%87%E8%96%84%E8%8D%B740%E7%B2%92%E7%93%B6%E8%A3%8556q%20%E7%93%B6%E8%A3%85&tab=all
output[6923450656181] 目录已存在
正在爬取: https://s.taobao.com/search?page=3&q=%E7%9B%8A%E8%BE%E6%9C%A8%E7%B3%96%E9%86%87%E8%96%84%E8%8D%B740%E7%B2%92%E7%93%B6%E8%A3%8556q%20%E7%93%B6%E8%A3%85&tab=all
output[6923450656181] 目录已存在
正在爬取: https://s.taobao.com/search?page=4&q=%E7%9B%8A%E8%BE%E6%9C%A8%E7%B3%96%E9%86%87%E8%96%84%E8%8D%B740%E7%B2%92%E7%93%B6%E8%A3%8556q%20%E7%93%B6%E8%A3%85&tab=all
```

```

跳转至详情页.....https://item.taobao.com/item
.htm?priceId=2147bf9b17175693187138769e1daa&id=672677478724&ns=1&abbucket=19#detail
向下滚动至-宝贝评价-元素可见.....
点击-宝贝评价.....
提取宝贝评价信息.....
跳转至详情页.....https://detail.tmall.com/item
.htm?priceId=2147bf9b17175693187138769e1daa&id=671052777903&ns=1&abbucket=19
向下滚动至-宝贝评价-元素可见.....
点击-宝贝评价.....
提取宝贝评价信息.....
跳转至详情页.....https://detail.tmall.com/item
.htm?priceId=2147bf9b17175693187138769e1daa&id=626907268822&ns=1&abbucket=19

```

[illegible]

1833 比超市便宜，还不错。

1834 一般不会给店家差评，无奈这家有些过分了 第一点是店家发错货了拍的101克70粒装的水糖醇居然会发90克40粒装。 第二点是快

1835 粉色不好吃，绿色噁噁噁

1836 买贵了

1837 不是给我买的 不知道

1838 日期很新，价格非常优惠，发货速度快，包装好哦，满意

1839 好丽友的好吃，不硬，味道也可以接受。价格也很划算。

1840 回购

1841 日期新鲜，包装完整。价格优惠。服务态度好，满意

1842 量少美味，没醉碾压

1843 是神剑不是绿剑，当时看打眼了，不过东西还行

1844 五星好评！很好吃！

1845 第六次买了。

1846 口味味道：完全是骗子，明明是个盗贼，请大家看清楚，这个不是绿箭，不是绿箭，这个也不是水糖醇噁了还牙疼.....而且吃起来味道

1847 口味味道：口感很差，没味道，被骗了，买时可得看好再买，很差劲儿。

1848 口味味道：以为是绿箭 结果是神剑 价钱都差不多 味道还难吃 差评

1849 不好吃，是仿的绿箭的牌子，下次购物需谨慎，别被文字游戏坑死

1850 商品分量：少

1851 口味味道：完全假的不能在假了。。。

1852 口味味道：垃圾

1853 这个不是绿箭，仿的，没吃，扔了。

1854 东西很一般，东西很一般

1855 上当了，太难吃了

1856 不耐嚼，一下子就没味道了.....便宜没好货😞😞

1857 感觉不如h以前的挺立偏软一点口感不如以前了

1858 看成绿箭了，就买了。味道有点怪，嚼个一分钟左右就没味道了

1859 我三十岁的年纪可能长了七十岁的牙齿，居然嚼不动这“神剑”牌口香糖

分析：通过模拟登录和 `chromedriver` 代理，我们能实现淘宝页面的登录，登录之后通过 `html` 元素定位和 `json` 解析，获取的商品信息搜索商品并爬取商品链接，再对每个商品链接进行商品评论的爬取。

6.3.3 商品评论情感分析

```
epoch:1, step:10, loss:0.6778324842453003
epoch:1, step:20, loss:0.6226712465286255
epoch:1, step:30, loss:0.5853778719902039
epoch:1, step:40, loss:0.5527936220169067
epoch:1, step:50, loss:0.5183973908424377
epoch:1, step:60, loss:0.5041297010231018
epoch:1, step:70, loss:0.5028290748596191
epoch:1, step:80, loss:0.4761556088924408
epoch:1, step:90, loss:0.4811314046382904
epoch:1, step:100, loss:0.4798413813114166
```

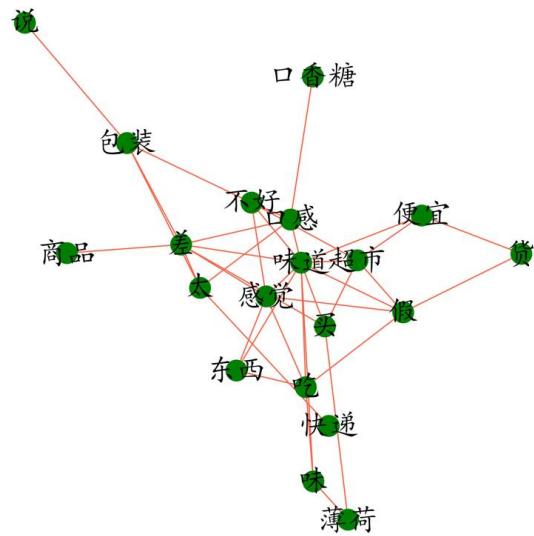
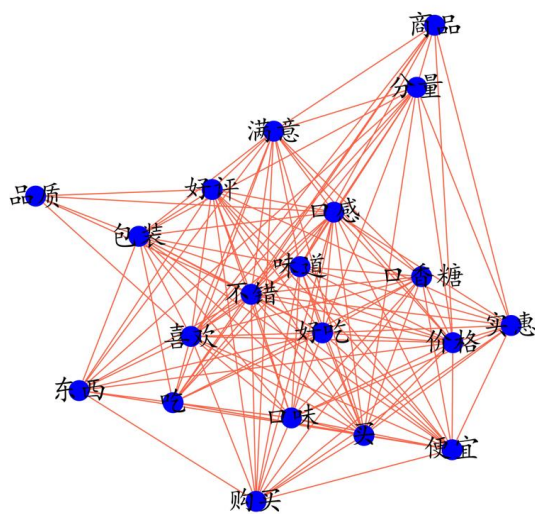
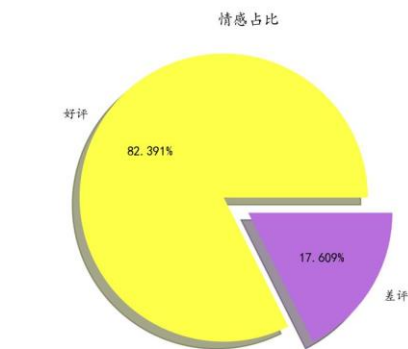
```
label_emotion,sentence
1,送的很快，口味丰富很好吃，能让嘴巴不寂寞
1,刚吃，清新口气，这个口味的我很喜欢！
1,很好吃
1,日期新鲜一切超满意预定支持赞赞赞
1,物美价廉，到货速度也快
1,还可以，跟超市一样吧，价格比超市的实惠！
0,在香港买不到的草本味
1,口感味道：非常不错👍
1,挺好的(∠・∠)∠ (∠・∠)∠ (∠・∠)∠ (∠・∠)∠ (∠・∠)∠ (∠・∠)∠ (∠・∠)∠ (∠・∠)∠ (∠・∠)∠ (∠・∠)∠
1,卖家服务态度很好，物流也很快，！
1,挺不错的
0,口感味道：👍
1,商品分量：好 口感味道：好
1,正品
0,口感味道：一不好
1,比便利店便宜多了
1,物美价廉
0,great
0,good
1,还没吃，但是包装可以，日期很好，快递员非常好，今天还下雨，送到
```

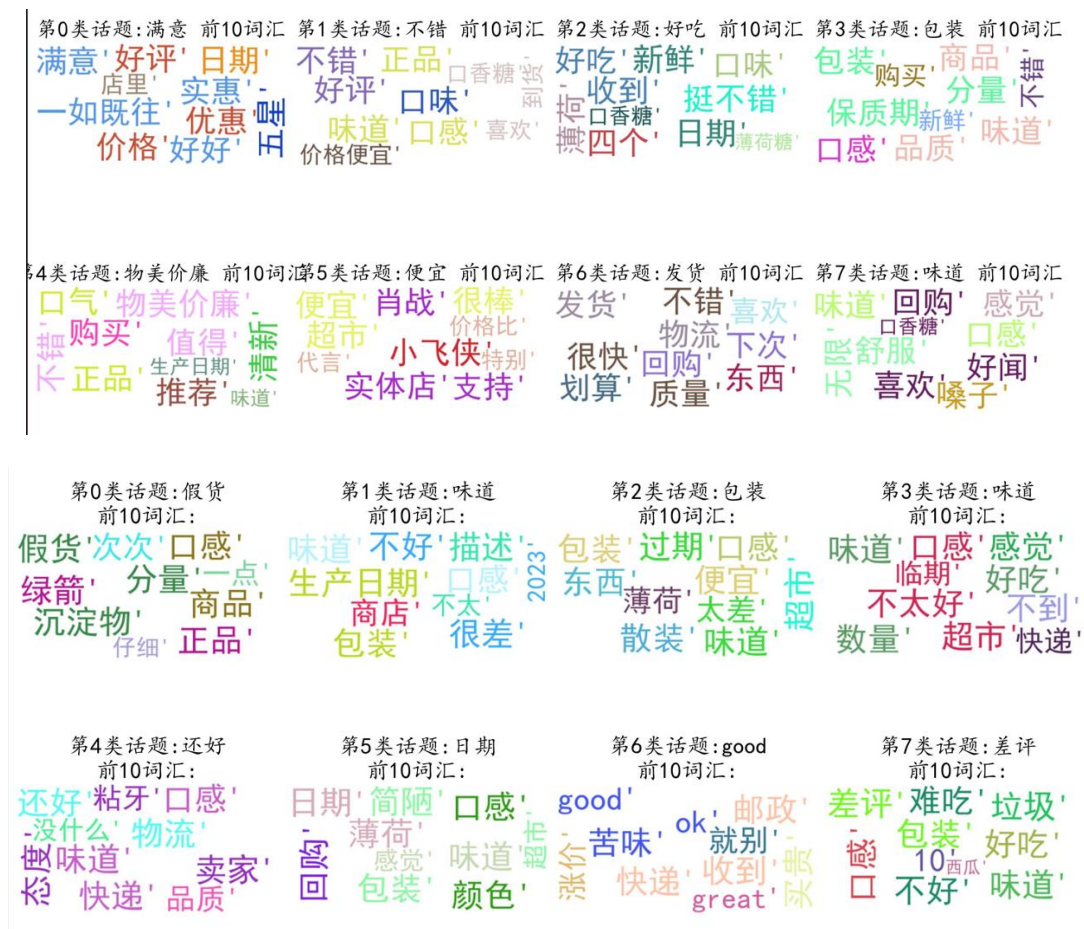
```
comment
1 送的很快，口味丰富很好吃，能让嘴巴不寂寞
2    刚吃，清新口气，这个口味的我很喜欢！
3      很好吃
4        日期新鲜一切超满意限定支持赞赞赞
5          物美价廉，到货速度也快
['送的很快，口味丰富很好吃，能让嘴巴不寂寞', '刚吃，清新口气，这个口味的我很喜欢！', '很好吃', '日期新鲜一切超满意限定支持赞赞赞', '物美价廉，到货速度也快']
F:\pythonProject\Product_Review_Analysis\output\6923450656181 目录已存在
Bert模型预测结束
cuda:0
['送的很快，口味丰富很好吃，能让嘴巴不寂寞', '刚吃，清新口气，这个口味的我很喜欢！', '很好吃', '日期新鲜一切超满意限定支持赞赞赞', '物美价廉，到货速度也快']
F:\pythonProject\Product_Review_Analysis
F:\pythonProject\Product_Review_Analysis\output\6923450656181 目录已存在
models\bert\onnx 目录已存在
Bert模型转onnx模型预测结束
```

分析：通过使用 weibo2018 评论数据集，对 Bert 的输出传入自定义的网络进行二分类微调，得到最优模型后转化为 ONNX 模型并进行推理。

6.3.4 评论数据分析

id	bel	emot	sentence	cutword
1	0		在香港胃才香港胃不到草本味	
2	0		口味味道:口味味道(合)	
3	0		口味味道:口味味道不好	
4	0	great	great	
5	0	good	good	
6	0		货不对版,货不对版拍明说明哪款收到货错	
7	0		这次的包装包装散12瓶	
8	0		包装太low包装太low随便烂塑料袋装散装	
9	0		本款口香糖本款口香糖入口越嚼越软如嚼稀泥理想口味味道不好	
10	1		日期还可日期	
11	0		假的吧,真假超市买味道颜色打开绿色前三口薄荷味一点薄荷味口味	
12	0		包装不一样包装不知真假	
13	0		感觉益达和感觉益达颗粒变小不知线下变小	
14	0		颜色不对颜色	
15	0		西瓜和冰棒西瓜冰棒好吃罗汉果金银花好吃	
16	0		天猫超市7天猫超市提供上门配送咨询客服客服说天猫订单支持上门	
17	0		阿里这里阿里没落原因忘记客户	
18	0		等了好久好久	
19	0		最难吃的口难吃口香糖硬夹橡胶算了牌子拉黑口味味道差劲太硬	
20	0		有标签在标签难看	
21	0		当你看到这条评论时对此产品评价满意想一份优质商品给出最终赞美	
22	0		还可以哦	
23	0		价廉物美价廉物美餐后在外刷牙时理想替代品容积变感觉净含量	
24	0		有点小贵小贵	
25	0		日期太临近日期太临近感觉不好	
26	0		买	
27	0		可以	
28	0		柠檬酸吃柠檬酸吃牙齿酸有没有含柠檬酸	
29	0		但凡客服客服态度好点生气	
30	0		第二次购买第二次购买	
31	0		讲真,不讲真太贪便宜假益达廉价胶沾牙嚼时间长发苦发涩光滑	
32	0		味道不好味道不好一两分钟味道正品谨慎购买	
33	0		日期旧,日期旧去年月旧	
34	0		日期有点旧日期旧	
35	0		这个蓝莓蓝莓味看实不算持久薄荷	
36	0		还可以。超市便宜	
37	0		买错口味买错口味	
38	0		图左面是图左面超市里买时寄包装扫码活动假	





分析: 首先通过已经进行感情分析的评论划分为好评和差评两类, 使用词云展现词频效果; 通过共现语义网络图展现每个高频词在整个商品评论语义关系; 最后通过 LDA 建模拟合进行评论话题的生成。

7 总结与展望

7.1 系统实现的总结

条形码检测与识别: 由于使用预训练模型, 效果比较好, 小模型识别速度也快。

商品信息评论爬取: 基本能够全自动获取指定商品信息的评论, 能够避免绝大部分反爬措施。

商品评论情感分析: 通过对 Bert 模型微调实现高效情感分析, 并且转化为 ONNX 模型, 使得能在其他环境下高效使用。

商品评论数据分析: 通过利用机器学习和统计学方法, 实现了共现语义网络和话题分析等, 挖取评论数据当中的潜在信息并且进行有效的可视化展示。

7.2 系统功能的展望

信息爬取: ip 容易被警告, 需要通过代理或者更换账号减少特殊反爬措施触发;

可以尝试 API 获取信息或者抓包技术。

感情分析：对于部分评论的感情最终又标点符号决定或者是抽象复杂语义运用的评论无法准确判断。

数据分析：共现语义网络在评论较少时候存在关联性弱的情况；生成的主题信息不够具体和清晰。

7.3 个人成长的规划

这门课将会大大提升我的个人代码水平和认知水平，在未来面试或者工作时候遇到相关的问题解决起来也不会感到无从入手，并且再次学习的成本也会大大下降，对于未来的发展方向也多了一个选择。

8 参考文献

- [1] 蓝色是天.Python 小项目:通过商品条形码查询商品信息 [EB/OL].CSDN.2023-08-13.https://blog.csdn.net/weixin_46043195/article/details/125794653
- [2] xiejava1018.Python 爬取淘宝商品评价信息实战 [EB/OL].CSDN.2024-03-16.<https://blog.csdn.net/fullbug/article/details/136766498>
- [3] dengxiuqi.机器学习对中文微博进行情感分析 [EB/OL].CSDN.2021-05-16.<https://github.com/dengxiuqi/WeiboSentiment>
- [4] 阡之尘埃.Python 数据分析案例 23——电商评论文本分析(LDA, 共现网络) [EB/OL].CSDN.2024-01-02.https://blog.csdn.net/weixin_46277779/article/details/129473272

9 附录

9.1 条形码检测识别

```
"""
条形码的检测识别
"""
class BarCodeDet:
    # pt 模型保存位置
    os.environ['TORCH_HOME'] = 'F:/pythonProject/Product_Review_Analysis/models/'
    def __init__(self, image_path):
        self.barCode = None
        self.image = None
        # 加载已经训练好的模型
        model = ssdlite320_mobilenet_v3_large(weights='SSDLite320_MobileNet_V3_Large_Weights.DEFAULT')
        model.eval()
        # 图像预处理函数
        transform = transforms.Compose([
            transforms.ToTensor()
        ])
        # 读取图像
        self.image = cv2.imread(image_path)
        # 将图像转换为 PyTorch 张量并添加批次维度
        input_tensor = transform(self.image).unsqueeze(0)
        # 使用模型进行目标检测
        with torch.no_grad():
            prediction = model(input_tensor)
        # 解析检测结果并绘制边界框
        for score, label, bbox in zip(prediction[0]['scores'], prediction[0]['labels'],
prediction[0]['boxes']):
            if score > 0.5 and label == 1: # 如果置信度大于 0.5 且标签为 1（表示检测到的是物体）
                x, y, w, h = map(int, bbox)
                cv2.rectangle(self.image, (x, y), (w, h), (0, 255, 0), 2)
        # 使用 ZBar 库来识别条形码
        barCodeRectangle = []
        gray_image = cv2.cvtColor(self.image, cv2.COLOR_BGR2GRAY)
        for barcode in decode(gray_image):
            data = barcode.data.decode('utf-8')
            x, y, w, h = barcode.rect
            barCodeRectangle.append(barcode.rect)
            cv2.rectangle(self.image, (x, y), (x + w, y + h), (255, 0, 0), 2)
            cv2.putText(self.image, data, (x, y - 60), cv2.FONT_HERSHEY_SIMPLEX, 2, (255,
```

```

0, 0), 6)

        self.barCode = data
    if self.barCode is not None:
        print("条形码: " + self.barCode)
    else:
        print("未找到条形码! 请让照片更清晰或者条形码更明显!")
# 显示图片
def show_dispicture(self):
    # 显示图片
    height, width, _ = self.image.shape
    # 创建一个窗口
    cv2.namedWindow('Image', cv2.WINDOW_NORMAL)
    # 调整窗口大小以适应图像
    cv2.resizeWindow('Image', width, height)
    cv2.imshow('Image', self.image)
    cv2.waitKey(0) # 等待按下任意键
    cv2.destroyAllWindows()

```

```

"""
条形码信息获取
开发日志:
:suggest: 天猫商品信息搜索有局限性, 改用中国商品信息搜索更加强大!
:bug: 获取验证码图片有时候是返回一个重定向链接, 导致无法识别
:speculate:
:solve:
"""

class BarCodeRec:
    def __init__(self, shop_id):
        self.shop_id = shop_id
        # self.path = os.path.abspath(os.path.dirname(sys.argv[0]))
        # print(self.path)
        self.headers = {
            'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/94.0.4606.81 Safari/537.36'
        }
        # json 化
    def parse_json(self, s):
        begin = s.find('{')
        end = s.rfind('}') + 1
        return json.loads(s[begin:end])
    # 创建目录
    def mkdir(self, path):

```

```

# 去除首位空格
path = path.strip()
# 去除尾部 \ 符号
path = path.rstrip("\\")
# 判断路径是否存在
isExists = os.path.exists(path)
# 判断结果
if not isExists:
    os.makedirs(path)
    logger.info(path + ' 创建成功')
    return True
else:
    # 如果目录存在则不创建，并提示目录已存在
    logger.info(path + ' 目录已存在')
    return False
# 爬取 "tiaoma.cnaidc.com" 来查找商品信息
def requestT1(self):
    url = 'http://tiaoma.cnaidc.com'
    s = requests.session()
    # 获取验证码
    img_data = s.get(url + '/index/verify.html?time=', headers=self.headers).content
    time.sleep(2)
    # print(img_data)
    with open('verification_code.png', 'wb') as v:
        v.write(img_data)
    # 解验证码
    ocr = ddddocr.DdddOcr()
    with open('verification_code.png', 'rb') as f:
        img_bytes = f.read()
    code = ocr.classification(img_bytes)
    logger.info('当前验证码为 ' + code)
    # 请求接口参数
    data = {"code": self.shop_id, "verify": code}
    resp = s.post(url + '/index/search.html', headers=self.headers, data=data)
    resp_json = self.parse_json(resp.text)
    logger.info(resp_json)
    # 判断是否查询成功
    if resp_json['msg'] == '查询成功' and resp_json['json'].get('code_img'):
        # 保存商品图片
        img_url = "
        if resp_json['json']['code_img'].find('http') == -1:

```

```

        img_url = url + resp_json['json']['code_img']
    else:
        img_url = resp_json['json']['code_img']
    try:
        shop_img_data = s.get(img_url, headers=self.headers, timeout=10, ).content
        # 新建目录
        self.mkdir("output" + '\\' + self.shop_id)
        localtime = time.strftime("%Y%m%d%H%M%S", time.localtime())
        # 保存图片
        with open("output" + '\\' + self.shop_id + '\\' + self.shop_id + '.png', 'wb') as v:
            v.write(shop_img_data)
        logger.info("output" + '\\' + self.shop_id + '\\' + self.shop_id + '.png')
    except requests.exceptions.ConnectionError:
        logger.info('访问图片 URL 出现错误! ')
    if resp_json['msg'] == '验证码错误':
        return self.requestT1()
    return resp_json

```

9.2 商品信息评论获取

"""

开发日志:

:issue: 登录界面的各种变化, 无法实现全自动解放双手

:issue: 网页版只能查看部分评论, 能否通过开发者工具中模拟安卓页面进行爬取

:issue: 爬取到一定情况下会弹出一个互动验证

:suggest: 这个模块可以封装的更好

:bug: liushen-6901294179165.jpg 的爬取中, 爬取了 48 个商品, 有三个广告商品

:speculate:

:solve:

:bug: yida-6923450656181.jpg 的爬取中, 在 45 个商品数据当中爬取了 46 个商品, 有一个多出来的商品并没有存在于网页当中, 但是商品信息与目标查商品信息匹配度为百分之 30~40

:speculate:

:solve:

"""

class ComInfo:

"""

先让 Chrome 浏览器进入调试状态的终端命令:

chrome.exe --remote-debugging-port=9222 --incognito

查询端口:

netstat -ano|findstr "9222"

```

"""
def __init__(self,
              barCode="6923450656181",
              scriptDirectory="F:\\pythonProject\\Product_Review_Analysis",
              KEYWORD='益达木糖醇薄荷 40 粒瓶装 56g 瓶装',
              cur_page=1,
              max_page=5,
              index_page_time=3):
    self.barCode = barCode
    # 获取项目位置
    self.scriptDirectory = scriptDirectory
    # 此处端口保持和命令行启动的端口一致
    self.chrome_options = Options()
    # 隐身模式
    self.chrome_options.add_argument("--incognito")
    # 设置的端口要和命令行启动的端口一致
    self.chrome_options.add_experimental_option("debuggerAddress", "localhost:9222")
    # 参数: --user-data-dir='F:/pythonProject/Product_Review_Analysis/data/userinfo/tb'
    self.chrome_options.add_argument(
        f"user-data-dir={self.scriptDirectory}\\data\\userinfo\\tb") # 实际脚本的目录下
的 userdata
    self.KEYWORD = KEYWORD
    # 爬取的页数
    self.cur_page = cur_page
    self.max_page = max_page
    self.index_page_time = index_page_time
def selenium_tb(self, option_fun):
    isLogging = False
    isLogging_ans = 0
    while isLogging is False and isLogging_ans < 3:
        isLogging = self.login_taobao()
    if isLogging:
        sleep(self.index_page_time)
        print('已经登录')
        # 新建目录
        self.mkdir("output" + '\\' + self.barCode)
        # 清空爬取商品信息的文件
        products_path = os.path.join(self.scriptDirectory, "output", self.barCode,
'products.csv')
        comments_path = os.path.join(self.scriptDirectory, "output", self.barCode,
'comments.csv')
        # print(products_path)

```

```
# print(comments_path)
if option_fun == 0:
    # 判断路径是否存在
    products_isExists = os.path.exists(products_path)
    if products_isExists:
        os.remove(products_path)
    url = 'https://s.taobao.com/search?page=1&q=' + quote(self.KEYWORD) +
'&tab=all'

    self.index_page(url, self.cur_page, self.max_page)
    comments_isExists = os.path.exists(comments_path)
    if comments_isExists:
        os.remove(comments_path)
    self.get_prod_comments()
else:
    comments_isExists = os.path.exists(comments_path)
    if comments_isExists:
        os.remove(comments_path)
    self.get_prod_comments()
print("爬取结束")
else:
    print("登录失败")
```

9.3 商品评论情感分析

```
# 模型结构
# 网络结构
class Net(nn.Module):
    def __init__(self, input_size):
        super(Net, self).__init__()
        self.fc = nn.Linear(input_size, 1)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        out = self.fc(x)
        out = self.sigmoid(out)
        return out

class BertOnnxPerdict:
    def __init__(self, scriptDirectory="F:\\pythonProject\\Product_Review_Analysis",
                 barCode="6923450656181",
                 BEST_MODEL_PATH="models/bert/finetune/bert_dnn_140.model",
                 MODEL_PATH="models/bert/chinese_wwm_pytorch",

path_comments="F:/pythonProject/Product_Review_Analysis/output/6923450656181/comments.csv",
                 path_commentsSentiment="F:\\pythonProject\\Product_Review_Analysis" +
"\output\\6923450656181\\commentsSentiment_onnx.csv",
                 ):
        self.scriptDirectory = scriptDirectory
        self.barCode = barCode
        # 加载微调好的 PyTorch 模型
        self.BEST_MODEL_PATH = BEST_MODEL_PATH
        self.net = torch.load(self.BEST_MODEL_PATH)
        self.net.eval()
        self.MODEL_PATH = MODEL_PATH
        self.tokenizer = BertTokenizer.from_pretrained(self.MODEL_PATH) # 分词器
        self.bert = BertModel.from_pretrained(self.MODEL_PATH) # 模型
        # 评论获取
        self.path_comments = path_comments
        self.df_comments = pd.read_csv(self.path_comments)
        # 删除包含 NaN 值的行
        self.df_comments.dropna(subset=['comment'], inplace=True)
        # df_comments.head(5)
        # df 转列表
        self.ls_comments = self.df_comments["comment"].tolist()
        print(self.ls_comments[:5])
        self.path_commentsSentiment = path_commentsSentiment
```

```

# 导出模型为 ONNX 格式
# dummy_input = (input_ids, attention_mask)
# 保存模型预测结果
# save_to_csv(products, os.path.join(self.scriptDirectory, "output", self.barCode,
'products.csv'),fieldnames=['url', 'title'])
# 开始模型预测
def bert_onnx_predicted(self):
    print(self.scriptDirectory)
    # 创建文件夹
    self.mkdir(self.scriptDirectory + '\\' + "output" + '\\' + self.barCode)
    # 保存评论预测的文件路径
    # commentsSentiment_path = os.path.join(scriptDirectory, "output", barCode,
'commentsSentiment.csv')
    # 判断路径是否存在
    commentsSentiment_isExists = os.path.exists(self.path_commentsSentiment)
    # 删除文件
    if commentsSentiment_isExists:
        os.remove(self.path_commentsSentiment)
    self.mkdir("models/bert/onnx")
    # 评论预测
    for s in self.ls_comments:
        if len(s) > 500:
            s = s[:500]
            tokens = self.tokenizer([s], padding=True)
            input_ids = torch.tensor(tokens["input_ids"]).to(self.device)
            attention_mask = torch.tensor(tokens["attention_mask"]).to(self.device)
            self.bert.to(self.device)
            # print("input_ids device:", input_ids.device)
            # print("attention_mask device:", attention_mask.device)
            # print("bert device:", bert.device)
            last_hidden_states = self.bert(input_ids, attention_mask=attention_mask)
            bert_output = last_hidden_states[0][:, 0]
            output_path = "models/bert/onnx/bert_dnn.onnx"
            torch.onnx.export(self.net, bert_output, output_path, opset_version=11)
            # 加载 ONNX 模型并进行预测
            onnx_model = onnx.load(output_path)
            onnx_session = onnxruntime.InferenceSession(output_path)
            input_data = {
                'onnx::Gemm_0': bert_output.cpu().detach().numpy()
            }
            onnx_outputs = onnx_session.run(None, input_data)
            # print(onnx_outputs)

```



```

outputs = self.net(bert_output)
# print(outputs)
# emotion_labels = ["积极情感" if item > 0.5 else "消极情感" for item in outputs]
# for sentence, emotion in zip([s], emotion_labels):
#     print(sentence,"->", emotion)
for label_emotion, sentence in zip([1 if item > 0.5 else 0 for item in outputs], [s]):
    # print(label_emotion, sentence)
    self.save_to_csv([{'label_emotion': label_emotion, 'sentence': sentence}],
                    self.path_commentsSentiment,
                    fieldnames=['label_emotion', 'sentence'])

```

9.4 商品评论数据分析

```

def commentsAnalysis(scriptDirectory, barCode,
                    path_stopwords,
                    path_commentsSentiment_onnx,
                    path_save_commentsAnalysis):
    plt.rcParams['font.sans-serif'] = ['KaiTi'] # 指定默认字体 SimHei 黑体
    plt.rcParams['axes.unicode_minus'] = False # 解决保存图像是负号'
    scriptDirectory = scriptDirectory
    barCode = barCode
    # 停用词
    stopwords = []
    path_stopwords = path_stopwords
    with open(path_stopwords, "r", encoding="utf8") as f:
        for w in f:
            stopwords.append(w.strip())
    # 已经分好类的评论
    path_commentsSentiment_onnx = path_commentsSentiment_onnx
    data_comments = pd.read_csv(path_commentsSentiment_onnx)
    df_comments = pd.DataFrame(data_comments)
    # 生成文件夹存储数据分析的结果文件
    path_save_commentsAnalysis = path_save_commentsAnalysis
    mkdir(path_save_commentsAnalysis)
    # Jieba 分词函数
    def txt_cut(sentence):
        lis = [word for word in jieba.lcut(sentence) if word not in stopwords]
        return " ".join(lis)

```

```

# 分词并且，添加分词列
df_comments['cutword'] = df_comments['sentence'].astype('str').apply(txt_cut)
df_comments.head(5)
def randomcolor():
    colorArr = ['1', '2', '3', '4', '5', '6', '7', '8', '9', 'A', 'B', 'C', 'D', 'E', 'F']
    color = "#" + ".join([random.choice(colorArr) for i in range(6)])
    return color
# [randomcolor() for i in range(3)]
plt.figure(figsize=(5, 5), dpi=180)
p1 = df_comments['label_emotion'].value_counts()
plt.pie(p1, labels=['好评', '差评'], autopct="%1.3f%%", shadow=True, explode=(0.2, 0),
        colors=[randomcolor() for i in range(2)]) # 带阴影，某一块里中心的距离
plt.title("情感占比")
plt.savefig(path_save_commentsAnalysis + "\\情感占比.jpg")
# plt.show()
# plt.close()
# 区分好评和差评
df_comments_good = df_comments[df_comments["label_emotion"] == 1].reset_index(drop=True)
df_comments_bad = df_comments[df_comments["label_emotion"] == 0].reset_index(drop=True)
writer_good = pd.ExcelWriter(path_save_commentsAnalysis + "\\comments_good.xlsx",
engine='xlsxwriter')
df_comments_good.to_excel(writer_good, index=False)
writer_good._save()
writer_bad = pd.ExcelWriter(path_save_commentsAnalysis + "\\comments_bad.xlsx",
engine='xlsxwriter')
df_comments_bad.to_excel(writer_bad, index=False)
writer_bad._save()
# df_comments_good.to_excel(path_save_commentsAnalysis + "\\comments_good.xlsx",
index=False)
# df_comments_bad.to_excel(path_save_commentsAnalysis + "\\comments_bad.xlsx",
index=False)
# df_comments_good.to_csv(path_save_commentsAnalysis+"\\comments_good.csv",
index=False,encoding="utf-8")
# df_comments_bad.to_csv(path_save_commentsAnalysis+"\\comments_bad.csv",
index=False,encoding="utf-8")
# 文本分析
# 词频分析
jieba.analyse.set_stop_words(path_stopwords)
# 合并一起
text_good = "
text_bad = "
for i in range(len(df_comments_good['cutword'])):

```

```

text_good += df_comments_good['cutword'][i] + '\n'
for i in range(len(df_comments_bad['cutword'])):
    text_bad += df_comments_bad['cutword'][i] + '\n'
j_r_good = jieba.analyse.extract_tags(text_good, topK=20, withWeight=True)
j_r_bad = jieba.analyse.extract_tags(text_bad, topK=20, withWeight=True)
df_wf_good = pd.DataFrame()
df_wf_bad = pd.DataFrame()
df_wf_good['word'] = [word[0] for word in j_r_good]
df_wf_bad['word'] = [word[0] for word in j_r_bad]
df_wf_good['frequency'] = [word[1] for word in j_r_good]
df_wf_bad['frequency'] = [word[1] for word in j_r_bad]
df_wf_good
df_wf_bad
# 生成词云
# Custom colour map based on Netflix palette
# mask = np.array(Image.open(scriptDirectory + '\\output\\' + barCode + '\\' + barCode + '.png'))
mask = np.array(Image.open(scriptDirectory + '\\data\\analysis\\money.png'))
cmap = matplotlib.colors.LinearSegmentedColormap.from_list("", [randomcolor() for i in
range(20)])
wordcloud = WordCloud(font_path="C:\\Windows\\Fonts\\simfang.ttf",
background_color='white', width=256, height=256,
                        colormap=cmap, max_words=100, mask=mask)
wordcloud_good = wordcloud.generate(text_good)
plt.figure(figsize=(10, 6), dpi=512)
plt.imshow(wordcloud_good, interpolation='bilinear')
plt.axis('off')
plt.tight_layout(pad=0)
plt.savefig(path_save_commentsAnalysis + '\\好评词云.jpg')
# plt.show()
# plt.close()
wordcloud_bad = wordcloud.generate(text_bad)
plt.figure(figsize=(10, 6), dpi=512)
plt.imshow(wordcloud_bad, interpolation='bilinear')
plt.axis('off')
plt.tight_layout(pad=0)
plt.savefig(path_save_commentsAnalysis + '\\差评词云.jpg')
# plt.show()
# plt.close()
# 共现语义网络
cut_word_list_good = list(map(lambda x: ".join(x), df_comments_good['cutword'].tolist()))
cut_word_list_bad = list(map(lambda x: ".join(x), df_comments_bad['cutword'].tolist()))
content_str_good = '.join(cut_word_list_good).split()
content_str_bad = '.join(cut_word_list_bad).split()

```

```

word_fre_good = pd.Series(tkinter._flatten(content_str_good)).value_counts() # 统计词频
word_fre_bad = pd.Series(tkinter._flatten(content_str_bad)).value_counts() # 统计词频
word_fre_good[:50]
word_fre_bad[:50]
keywords_good = word_fre_good[:50].index
keywords_bad = word_fre_bad[:50].index
keywords_good
keywords_bad
# 计算矩阵
matrix_good = np.zeros((len(keywords_good) + 1) * (len(keywords_good) + 1))
matrix_good = matrix_good.reshape(len(keywords_good) + 1, len(keywords_good) +
1).astype(str)
matrix_good[0][0] = np.NaN
matrix_good[1:, 0] = matrix_good[0, 1:] = keywords_good
matrix_good
cont_list_good = [cont.split() for cont in cut_word_list_good]
for i, w1 in enumerate(word_fre_good[:30].index):
    for j, w2 in enumerate(word_fre_good[:30].index):
        count = 0
        # 遍历表中的前五十个词之间在全部文档的关联性
        for cont in cont_list_good:
            if w1 in cont and w2 in cont:
                # 一个词无论在什么文本跟自己的关联肯定最大，其次是相邻一个位置的
                # 词，满足这两个条件则是共现一次
                # 数值越大代表这个两个词一起出现的概率很大，而且可能出现次数也
                # if abs(cont.index(w1) - cont.index(w2)) == 0 or abs(cont.index(w1) -
                # cont.index(w2)) == 1:
                if abs(cont.index(w1) - cont.index(w2)) == 1:
                    count += 1
                matrix_good[i + 1][j + 1] = count
matrix_bad = np.zeros((len(keywords_bad) + 1) * (len(keywords_bad) + 1))
matrix_bad = matrix_bad.reshape(len(keywords_bad) + 1, len(keywords_bad) + 1).astype(str)
matrix_bad[0][0] = np.NaN
matrix_bad[1:, 0] = matrix_bad[0, 1:] = keywords_bad
matrix_bad
cont_list_bad = [cont.split() for cont in cut_word_list_bad]
for i, w1 in enumerate(word_fre_bad[:20].index):
    for j, w2 in enumerate(word_fre_bad[:20].index):
        count = 0
        for cont in cont_list_bad:
            if w1 in cont and w2 in cont:

```

```

        # if abs(cont.index(w1) - cont.index(w2)) == 0 or abs(cont.index(w1) -
cont.index(w2)) == 1:
            if abs(cont.index(w1) - cont.index(w2)) == 1:
                count += 1
            matrix_bad[i + 1][j + 1] = count

# 存储
# kwdata = pd.DataFrame(data=matrix)
# kwdata.to_csv('关键词共现矩阵.csv', index=False, header=None, encoding='utf-8-sig')

# 查看
kwdata_good = pd.DataFrame(data=matrix_good[1:, 1:], index=matrix_good[1:, 0],
columns=matrix_good[0, 1:])
kwdata_bad = pd.DataFrame(data=matrix_bad[1:, 1:], index=matrix_bad[1:, 0],
columns=matrix_bad[0, 1:])

# 画图
plt.figure(figsize=(7, 7), dpi=512)
graph1_good = nx.from_pandas_adjacency(kwdata_good.iloc[:20, 0:20].astype(int))
nx.draw(graph1_good, with_labels=True, node_color='blue', font_size=25, edge_color='tomato')
plt.savefig(path_save_commentsAnalysis + '\\共现网络图-好评.jpg')
# plt.show()
# plt.close()
plt.figure(figsize=(7, 7), dpi=512)
graph1_bad = nx.from_pandas_adjacency(kwdata_bad.iloc[:20, 0:20].astype(int))
nx.draw(graph1_bad, with_labels=True, node_color='green', font_size=25, edge_color='tomato')
plt.savefig(path_save_commentsAnalysis + '\\共现网络图-差评.jpg')
# plt.show()
# plt.close()

# 话题分析
"""好评"""
# Tf-idf 分析，词频逆文档频率
# 文本转化为词向量
tf_vectorizer = TfidfVectorizer()
# tf_vectorizer = TfidfVectorizer(ngram_range=(2,2)) #2 元词袋
X_good = tf_vectorizer.fit_transform(df_comments_good.cutword)
# print(tf_vectorizer.get_feature_names_out())
print(X_good.shape)
# 查看高频词
data1_good = {'word': tf_vectorizer.get_feature_names_out(),
               'tfidf': X_good.toarray().sum(axis=0).tolist()}
df2_good = pd.DataFrame(data1_good).sort_values(by="tfidf", ascending=False,
ignore_index=True)
df2_good.head(20)

```

```

# LDA 建模，构建模型，并且拟合
n_topics_good = 8 # 需要生成的主题数量，这里是八类
lda_good = LatentDirichletAllocation(n_components=n_topics_good, max_iter=100,
                                     learning_method='batch',
                                     learning_offset=100,
                                     # doc_topic_prior=0.1,
                                     # topic_word_prior=0.01,
                                     random_state=0)

lda_good.fit(X_good)
# 查看结果函数
def print_top_words(model, feature_names, n_top_words):
    tword = []
    tword2 = []
    tword3 = []
    for topic_idx, topic in enumerate(model.components_):
        print("Topic #%d:" % topic_idx)
        topic_w = [feature_names[i] for i in topic.argsort()[: -n_top_words - 1:-1]]
        topic_pro = [str(round(topic[i], 3)) for i in topic.argsort()[: -n_top_words - 1:-1]] #
    (round(topic[i],3))
        tword.append(topic_w)
        tword2.append(topic_pro)
        print(" ".join(topic_w))
        print(" ".join(topic_pro))
        print(' ')
        word_pro = dict(zip(topic_w, topic_pro))
        tword3.append(word_pro)
    return tword3
# 输出每个主题对应词语和概率
feature_names = tf_vectorizer.get_feature_names_out()
n_top_words_good = 10
word_pro_good = print_top_words(lda_good, feature_names, n_top_words_good)
# 每个评论被分类到哪个主题
topics_good = lda_good.transform(X_good)
topics_good = np.argmax(topics_good, axis=1)
df_comments_good['topic'] = topics_good
# df.to_excel("data_topic.xlsx",index=False)
print(topics_good.shape)
print(df_comments_good.head(5))
# 通过词云图可视化函数
def generate_wordcloud(tup):
    color_list = [randomcolor() for i in range(10)] # 随机生成 10 个颜色
    wordcloud = WordCloud(background_color='white', font_path='simhei.ttf', # mask = mask,

```

```

#形状设置
max_words=10, max_font_size=50, random_state=42,
colormap=colors.ListedColormap(color_list) # 颜色
).generate(str(tup))

return wordcloud

# 词云显示
dis_cols = 4 # 一行几个
dis_rows = 3
dis_wordnum = 10
plt.figure(figsize=(5 * dis_cols, 5 * dis_rows), dpi=128)
kind_good = len(df_comments_good['topic'].unique())
for i in range(kind_good):
    ax = plt.subplot(dis_rows, dis_cols, i + 1)
    theme_title = list(word_pro_good[i].keys())[0]
    most10 = [(k, float(v)) for k, v in word_pro_good[i].items()][:dis_wordnum] # 高频词
    ax.imshow(generate_wordcloud(most10, interpolation="bilinear"))
    ax.axis('off')
    ax.set_title("第{}类话题: {} 前 {} 词汇".format(i, theme_title, dis_wordnum), fontsize=30)
plt.tight_layout()
plt.savefig(path_save_commentsAnalysis + '\\好评话题词云.jpg')
# plt.show()
# plt.close()
"""差评"""
# Tf-idf 分析, 词频逆文档频率
# 文本转化为词向量
tf_vectorizer = TfidfVectorizer()
# tf_vectorizer = TfidfVectorizer(ngram_range=(2,2)) #2 元词袋
X_bad = tf_vectorizer.fit_transform(df_comments_bad.cutword)
# print(tf_vectorizer.get_feature_names_out())
print(X_bad.shape)
# 查看高频词
data1_bad = {'word': tf_vectorizer.get_feature_names_out(),
              'tfidf': X_bad.toarray().sum(axis=0).tolist()}
df2_bad = pd.DataFrame(data1_bad).sort_values(by="tfidf", ascending=False,
ignore_index=True)
df2_bad.head(20)
# LDA 建模, 构建模型, 并且拟合
n_topics_bad = 8 # 需要生成的主题数量, 这里是八类
lda_bad = LatentDirichletAllocation(n_components=n_topics_bad, max_iter=100,
learning_method='batch',
learning_offset=100,
# doc_topic_prior=0.1,
# topic_word_prior=0.01,

```

```

random_state=0)

lda_bad.fit(X_bad)
# 查看结果函数
def print_top_words(model, feature_names, n_top_words):
    tword = []
    tword2 = []
    tword3 = []
    for topic_idx, topic in enumerate(model.components_):
        print("Topic #%d:" % topic_idx)
        topic_w = [feature_names[i] for i in topic.argsort()[:n_top_words - 1:-1]]
        topic_pro = [str(round(topic[i], 3)) for i in topic.argsort()[:n_top_words - 1:-1]] #
(round(topic[i],3))
        tword.append(topic_w)
        tword2.append(topic_pro)
        print(" ".join(topic_w))
        print(" ".join(topic_pro))
        print(' ')
        word_pro = dict(zip(topic_w, topic_pro))
        tword3.append(word_pro)
    return tword3
# 输出每个主题对应词语和概率
feature_names = tf_vectorizer.get_feature_names_out()
n_top_words_bad = 10
word_pro_bad = print_top_words(lda_bad, feature_names, n_top_words_bad)
# 每个评论被分类到哪个主题
topics_bad = lda_bad.transform(X_bad)
topics_bad = np.argmax(topics_bad, axis=1)
df_comments_bad['topic'] = topics_bad
# df.to_excel("data_topic.xlsx", index=False)
print(topics_bad.shape)
print(df_comments_bad.head(5))
# 词云显示
dis_cols = 4 # 一行几个
dis_rows = 3
dis_wordnum = 10
plt.figure(figsize=(5 * dis_cols, 5 * dis_rows), dpi=128)
kind_bad = len(df_comments_bad['topic'].unique())
for i in range(kind_bad):
    ax = plt.subplot(dis_rows, dis_cols, i + 1)
    theme_title = list(word_pro_bad[i].keys())[0]
    most10 = [(k, float(v)) for k, v in word_pro_bad[i].items()][:dis_wordnum] # 高频词
    ax.imshow(generate_wordcloud(most10, interpolation="bilinear"))
    ax.axis('off')

```



```
ax.set_title("第{}类话题:{}\n前{}词汇:".format(i, theme_title, dis_wordnum), fontsize=30)
plt.tight_layout()
plt.savefig(path_save_commentsAnalysis + '\\差评话题词云.jpg')
```